

cellline benchmarking

Qun Li

18 Aug 2025 at 15:50:59

Contents

1	Env	1
2	Libraries	1
3	Load Data	3
4	processing	3
5	selectedCelllines	5
6	MCF7MutSim	7
7	How to generate MCF7_merge_exp.rds	8
8	MCF7ExpSim	11
9	sessionInfo	13

1 Env

```
rm(list = ls())  
setwd("/Users/liqun/Desktop/scMutrace-tutorial/ReproducibilityAnalysis/Figure2/")
```

2 Libraries

```
library(pheatmap)  
library(ggplot2)  
library(circlize)  
## =====  
## circlize version 0.4.16
```

```

## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize\_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## =====
library(dplyr)
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
library(Seurat)
## Loading required package: SeuratObject
## Loading required package: sp
## 'SeuratObject' was built under R 4.5.0 but the current version is
## 4.5.1; it is recommended that you reinstall 'SeuratObject' as the ABI
## for R may have changed
##
## Attaching package: 'SeuratObject'
## The following objects are masked from 'package:base':
##
##   intersect, t
library(ComplexHeatmap)
## Loading required package: grid
## =====
## ComplexHeatmap version 2.24.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##   genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## =====
## ! pheatmap() has been masked by ComplexHeatmap::pheatmap(). Most of the arguments
##   in the original pheatmap() are identically supported in the new function. You

```

```
## can still use the original function by explicitly calling pheatmap::pheatmap().
##
## Attaching package: 'ComplexHeatmap'
## The following object is masked from 'package:pheatmap':
##
## pheatmap
```

3 Load Data

```
cellLine_database <- readRDS("../Data/HumanCellline/CellLine_Database_res.rds")
cellLine_mutation <- readRDS("../Data/HumanCellline/CellLine_Mutation_res.rds")
cellLine_annotation <- read.csv("../Data/HumanCellline/CellLine_Annotation.csv", header = T, row.names = "cellLineName")
# remove KPL1 (SNP fingerprinting confirms that this line is an MCF-7 derivative. DepMap metadata has b
cellLine_annotation <- cellLine_annotation[!cellLine_annotation$cellLineName %in% c("KPL1"),]

cellLine_mutation$MCF7_MayC9 <- cellLine_mutation$MCF7_MayC9 %>%
  mutate(position_cor = sub("_[ACGT]+_[ACGT]+$", "", position_cor))
cellLine_mutation$MCF7_NovC9 <- cellLine_mutation$MCF7_NovC9 %>%
  mutate(position_cor = sub("_[ACGT]+_[ACGT]+$", "", position_cor))
cellLine_mutation$MCF7_Parental <- cellLine_mutation$MCF7_Parental %>%
  mutate(position_cor = sub("_[ACGT]+_[ACGT]+$", "", position_cor))
cellLine_mutation$MCF7_WT3 <- cellLine_mutation$MCF7_WT3 %>%
  mutate(position_cor = sub("_[ACGT]+_[ACGT]+$", "", position_cor))
cellLine_mutation$MCF7_WT4 <- cellLine_mutation$MCF7_WT4 %>%
  mutate(position_cor = sub("_[ACGT]+_[ACGT]+$", "", position_cor))
cellLine_mutation$MCF7_WT5 <- cellLine_mutation$MCF7_WT5 %>%
  mutate(position_cor = sub("_[ACGT]+_[ACGT]+$", "", position_cor))
```

4 processing

```
ann_data_new = data.frame(Groups=cellLine_annotation$ann)
rownames(ann_data_new) <- rownames(cellLine_annotation)
ann_colors = list(Groups = c(Bone="black", Breast="#ff7f00", Cervix = "#33a02c", Stomach = "#cab2d6",
  Liver = "#a6cee3", Lung = "#e31a1c", Kidney = "#6a3d9a",
  Skin = "#b15928", Brain = "#fdbf6f", Normal = "#ffff99"))

data_cell_lines <- do.call("rbind", cellLine_database)
# remove KPL1 (SNP fingerprinting confirms that this line is an MCF-7 derivative. DepMap metadata has b
data_cell_lines <- data_cell_lines[!data_cell_lines$cellLineName %in% c("KPL1"),]

data_scMut_lines <- do.call("rbind", cellLine_mutation)

scMut_lines <- unique(data_scMut_lines$cellLineName)
cellLine_Names <- unique(data_cell_lines$cellLineName)

data_sim_scmut <- as.data.frame(matrix(nrow = length(scMut_lines), ncol = length(cellLine_Names)))
rownames(data_sim_scmut) <- scMut_lines
```

```

colnames(data_sim_scmut) <- cellLine_Names

for (i in 1:length(cellLine_Names)){
  cellLine_name_tmp <- colnames(data_sim_scmut)[i]
  cellLine_name_tmp_pos <- data_cell_lines[data_cell_lines$cellLineName == cellLine_name_tmp,]$position
  for(j in 1:length(scMut_lines)){
    scLine_name_tmp <- rownames(data_sim_scmut)[j]
    scmut_pos <- data_scMut_lines[data_scMut_lines$cellLineName == scLine_name_tmp,]$position_cor
    score_tmp <- length(intersect(scmut_pos, cellLine_name_tmp_pos)) / length(union(scmut_pos, cellLine
    data_sim_scmut[j, i] <- score_tmp
  }
}

data_scaled <- t(scale(t(as.matrix(data_sim_scmut))))

row_names_order <- c("KATOIII", "HGC1", "HGC2", "NCIN87", "SNU161", "SNU162", "NUGC4",
  "SNU638", "MKN45", "SNU601_ATAC", "SNU601_RNA", "SNU668",
  "MCF7_Parental", "MCF7_WT3", "MCF7_WT4", "MCF7_WT5", "MCF7_MayC9", "MCF7_NovC9")

data_scaled <- data_scaled[row_names_order,]

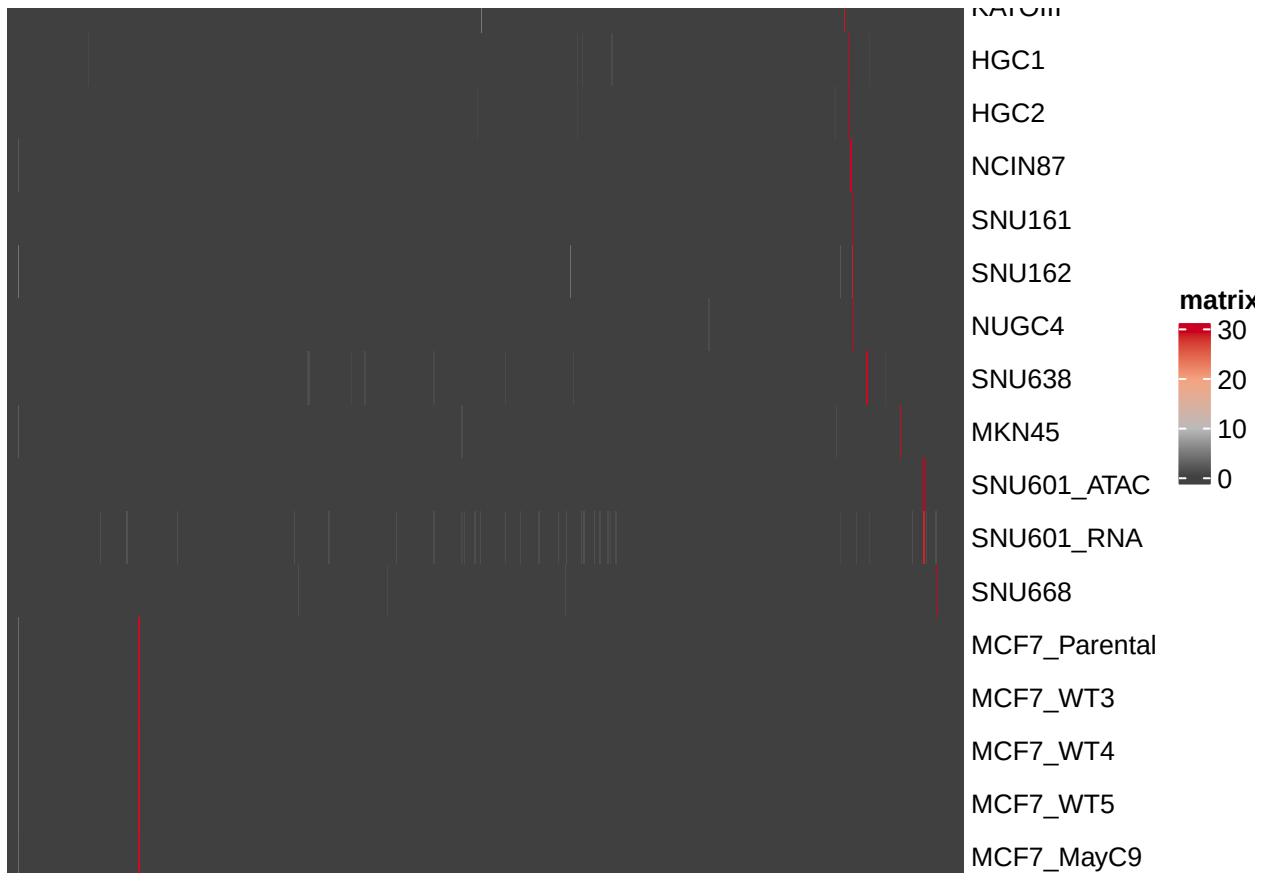
max_val <- max(data_scaled)
min_val <- min(data_scaled)
breaks <- seq(min_val, max_val, length.out = 4)

col_fun_v1 <- circlize::colorRamp2(
  breaks,
  c("#404040", "#bababa", "#f4a582", "#ca0020")
)

p_total <- pheatmap(data_scaled,
  cluster_rows = FALSE, cluster_cols = FALSE,
  annotation_col = ann_data_new,
  annotation_colors = ann_colors,
  color = col_fun_v1,
  cellheight = 20, cellwidth = 0.5, show_colnames = FALSE)
## Warning: Column annotation has different order from matrix columns. Adjust the
## column annotation based on column names of the matrix.

p_total

```



5 selectedCelllines

```
marked_celllines <- c("HGC27", "KATOIII", "MKN45", "NCIN87", "NUGC4", "SNU16", "SNU601", "SNU638", "SNU668")

normalCellLines <- unique(cellLine_database$Normal$cellLineName)

stomachCellLines <- unique(cellLine_database$Stomach$cellLineName)[unique(cellLine_database$Stomach$cellLineName) %in% marked_celllines]

selected_cellLines <- c("A549", "JHU028", "LK2",
                        "NOS1", "CHLA10", "U251MG", "U251MGDM", "MCF7", "T47D",
                        "HELA", "HOKUG", "7860", "ACHN", "HEPG2", "HUH7",
                        "SKMEL5", "MEWO",
                        #"SKGT2", "HGC27", "KATOIII", "MKN45", "NCIN87", "NUGC4", "SNU16", "SNU601", "SNU638",
                        normalCellLines, stomachCellLines)

ann_data_new = data.frame(Groups=cellLine_annotation$ann)

rownames(ann_data_new) <- rownames(cellLine_annotation)

ann_colors = list(Groups = c(Bone="black", Breast="#ff7f00", Cervix = "#33a02c", Stomach = "#cab2d6",
                             Liver = "#a6cee3", Lung = "#e31a1c", Kidney = "#6a3d9a",
                             Skin = "#b15928", Brain = "#fdbf6f", Normal = "#ffff99"))
```

```

data_sim_scmut_selected <- as.data.frame(matrix(nrow = length(scMut_lines), ncol = length(selected_cellLines),
rownames(data_sim_scmut_selected) <- scMut_lines
colnames(data_sim_scmut_selected) <- selected_cellLines

for (i in 1:length(selected_cellLines)){
  cellLine_name_tmp <- colnames(data_sim_scmut_selected)[i]
  cellLine_name_tmp_pos <- data_cell_lines[data_cell_lines$cellLineName == cellLine_name_tmp,]$position
  for(j in 1:length(scMut_lines)){
    scLine_name_tmp <- rownames(data_sim_scmut_selected)[j]
    scmut_pos <- data_scMut_lines[data_scMut_lines$cellLineName == scLine_name_tmp,]$position_cor
    score_tmp <- length(intersect(scmut_pos, cellLine_name_tmp_pos)) / length(union(scmut_pos, cellLine_name_tmp_pos))
    data_sim_scmut_selected[j, i] <- score_tmp
  }
}

data_scaled_selected <- t(scale(t(as.matrix(data_sim_scmut_selected))))

data_scaled_selected <- data_scaled_selected[row_names_order,]

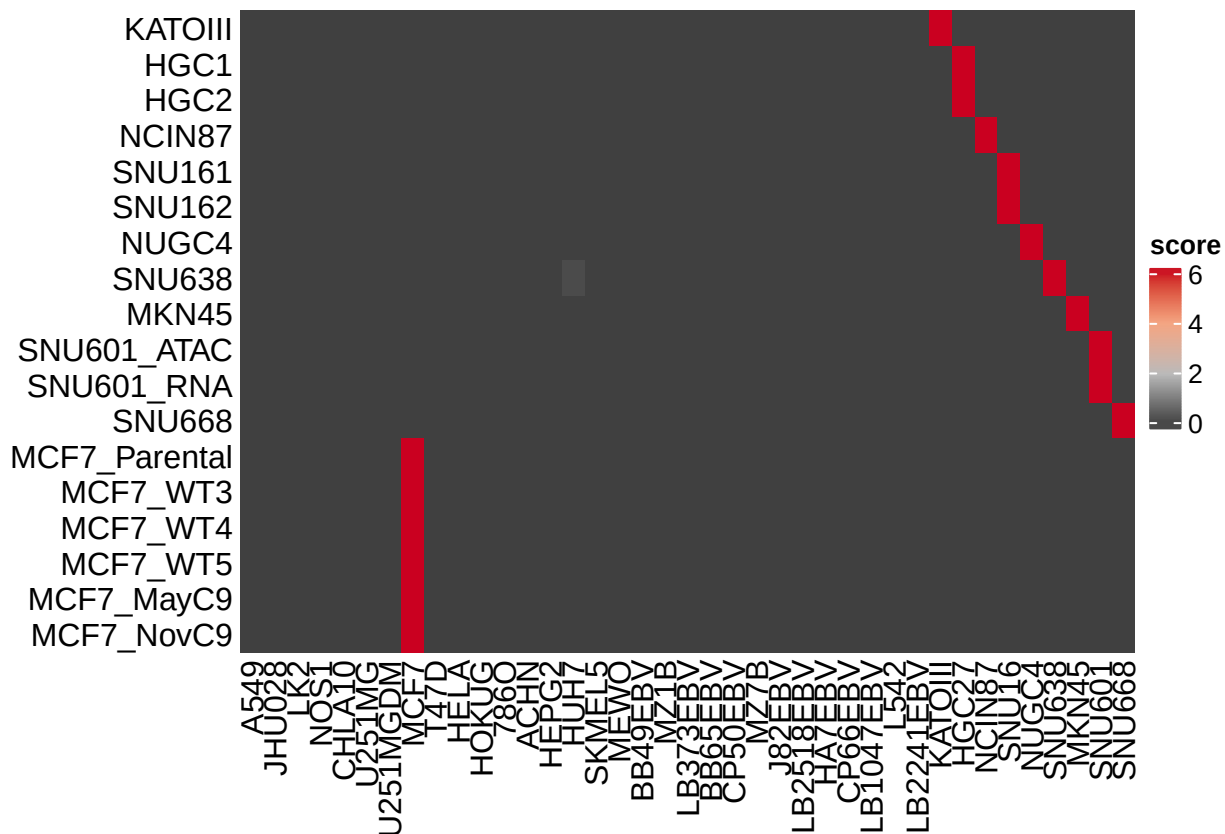
max_val <- max(data_scaled_selected)
min_val <- min(data_scaled_selected)
breaks <- seq(min_val, max_val, length.out = 4)

col_fun_v1 <- circlize::colorRamp2(
  breaks,
  c("#404040", "#bababa", "#f4a582", "#ca0020")
)

p_selected <- Heatmap(data_scaled_selected,
  name = "score",
  col = col_fun_v1,
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  show_column_names = TRUE,
  row_names_side = "left",
  column_names_side = "bottom")

p_selected

```



6 MCF7MutSim

```

MCF7_colnames <- c("MCF7_MayC9", "MCF7_NovC9", "MCF7_Parental", "MCF7_WT3", "MCF7_WT4", "MCF7_WT5")
data_sim_scmut_MCF7 <- as.data.frame(matrix(nrow = length(MCF7_colnames), ncol = length(MCF7_colnames)))
rownames(data_sim_scmut_MCF7) <- MCF7_colnames
colnames(data_sim_scmut_MCF7) <- MCF7_colnames

for (i in 1:length(MCF7_colnames)){
  cellLine_name_tmp <- colnames(data_sim_scmut_MCF7)[i]
  cellLine_name_tmp_pos <- data_scmut_lines[data_scmut_lines$cellLineName == cellLine_name_tmp,]$position
  for(j in 1:length(MCF7_colnames)){
    scLine_name_tmp <- rownames(data_sim_scmut_MCF7)[j]
    scmut_pos <- data_scmut_lines[data_scmut_lines$cellLineName == scLine_name_tmp,]$position_cor
    score_tmp <- length(intersect(scmut_pos, cellLine_name_tmp_pos)) / length(union(scmut_pos, cellLine_name_tmp_pos))
    data_sim_scmut_MCF7[j, i] <- score_tmp
  }
}

data_scaled_MCF7 <- t(scale(t(as.matrix(data_sim_scmut_MCF7))))

max_val <- max(data_sim_scmut_MCF7)
min_val <- min(data_sim_scmut_MCF7)

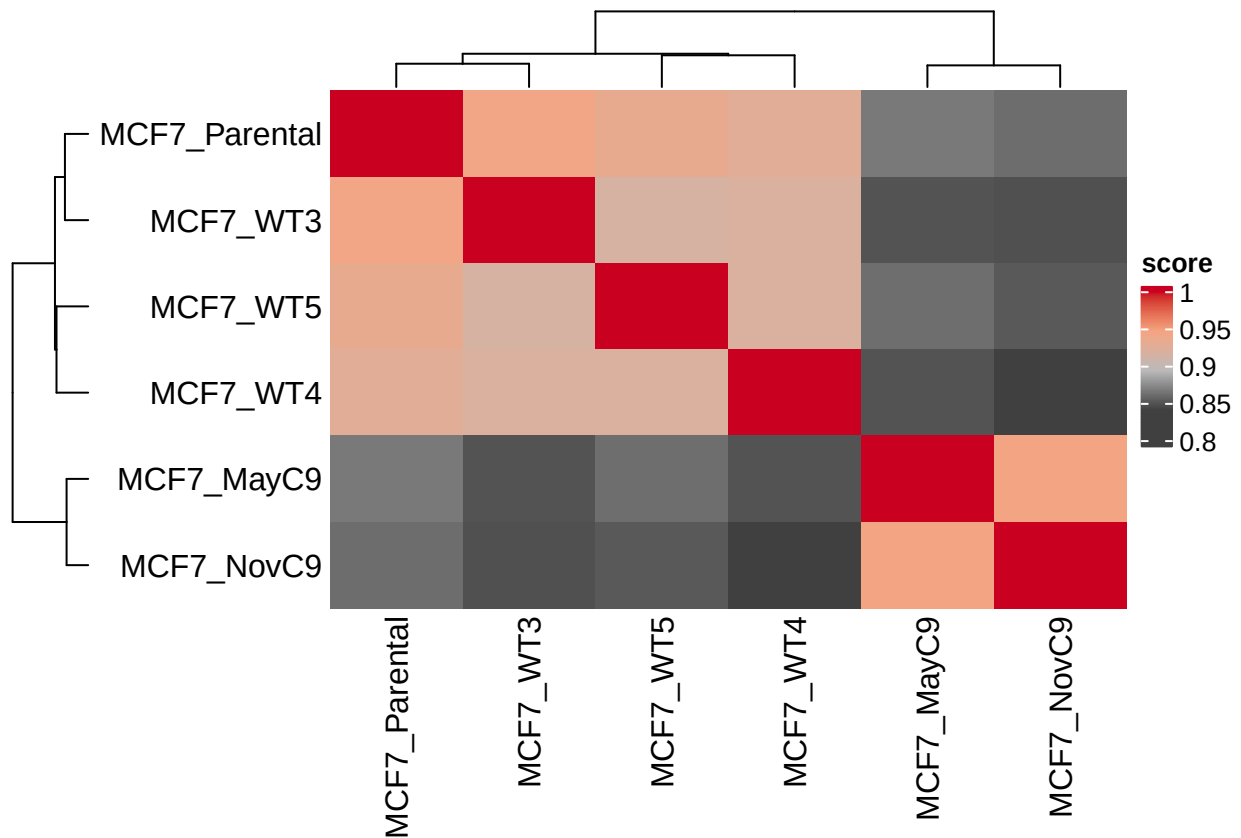
```

```
breaks <- seq(min_val, max_val, length.out = 4)

col_fun_v1 <- circlize::colorRamp2(
  breaks,
  c("#404040", "#bababa", "#f4a582", "#ca0020")
)

p_selected_MCF7 <- Heatmap(data_sim_scmut_MCF7,
  name = "score",
  col = col_fun_v1,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  show_column_names = TRUE,
  row_names_side = "left",
  column_names_side = "bottom")
## Warning: The input is a data frame-like object, convert it to a matrix.

p_selected_MCF7
```



7 How to generate MCF7_merge_exp.rds

```
# load data
MCF7_Parental <- Read10X("../data/exp/MCF7_Parent/") # note the folder name is "MCF7_Parent", output of
```



```

MCF7_WT3 <- Read10X("../data/exp/MCF7_WT3/") # note the folder name is "MCF7_WT3", output of cell range
MCF7_WT4 <- Read10X("../data/exp/MCF7_WT4/") # note the folder name is "MCF7_WT4", output of cell range
MCF7_WT5 <- Read10X("../data/exp/MCF7_WT5/") # note the folder name is "MCF7_WT5", output of cell range

MCF7_May <- Read10X("../data/exp/MCF7_MayC9/") # note the folder name is "MCF7_MayC9", output of cell range
MCF7_Nov <- Read10X("../data/exp/MCF7_NovC9/") # note the folder name is "MCF7_NovC9", output of cell range

MCF7_Parental <- CreateSeuratObject(MCF7_Parental, project = "MCF7_Parental")
MCF7_WT3 <- CreateSeuratObject(MCF7_WT3, project = "MCF7_WT3")
MCF7_WT4 <- CreateSeuratObject(MCF7_WT4, project = "MCF7_WT4")
MCF7_WT5 <- CreateSeuratObject(MCF7_WT5, project = "MCF7_WT5")

MCF7_May <- CreateSeuratObject(MCF7_May, project = "MCF7_May")
MCF7_Nov <- CreateSeuratObject(MCF7_Nov, project = "MCF7_Nov")
# filter
# Cells with either <1,000 detected genes or >0.15 mitochondrial fraction were excluded from further analysis
# Finally, the resulting expression matrix was filtered to remove genes detected in <3 cells.
filter_scRNA_sample <- function(obj,
                                min_genes = 1000,
                                max_mt = 15,
                                min_cells_per_gene = 3,
                                mt_pattern = "^MT-") {

  stopifnot("Seurat" %in% class(obj))

  obj[["percent.mt"]] <- PercentageFeatureSet(obj, pattern = mt_pattern)

  obj <- subset(obj, subset = nFeature_RNA >= min_genes & percent.mt <= max_mt)

  counts_mat <- GetAssayData(obj, assay = "RNA", slot = "counts")

  gene_filter <- Matrix::rowSums(counts_mat > 0) >= min_cells_per_gene

  obj <- obj[which(gene_filter), ]

  return(obj)
}

MCF7_Parental <- filter_scRNA_sample(MCF7_Parental)
MCF7_WT3 <- filter_scRNA_sample(MCF7_WT3)
MCF7_WT4 <- filter_scRNA_sample(MCF7_WT4)
MCF7_WT5 <- filter_scRNA_sample(MCF7_WT5)
MCF7_May <- filter_scRNA_sample(MCF7_May)
MCF7_Nov <- filter_scRNA_sample(MCF7_Nov)

MCF7_Parental$sample <- "Parental"
MCF7_WT3$sample <- "WT3"
MCF7_WT4$sample <- "WT4"
MCF7_WT5$sample <- "WT5"
MCF7_May$sample <- "May"
MCF7_Nov$sample <- "Nov"

# merge

```

```

MCF7_Merge <- merge(MCF7_Parental, y = c(MCF7_WT3, MCF7_WT4, MCF7_WT5, MCF7_May, MCF7_Nov), add.cell.id

MCF7_Merge <- NormalizeData(MCF7_Merge)
MCF7_Merge <- FindVariableFeatures(MCF7_Merge)
MCF7_Merge <- ScaleData(MCF7_Merge)
MCF7_Merge <- RunPCA(MCF7_Merge)
MCF7_Merge <- RunUMAP(MCF7_Merge, dims = 1:20)
MCF7_Merge <- RunTSNE(MCF7_Merge, dims = 1:20)

my_colors <- c(
  "May"      = "#D73027",
  "Nov"      = "#FC8D59",
  "Parental" = "#7F7F7F",
  "WT3"      = "#4575B4",
  "WT4"      = "#74ADD1",
  "WT5"      = "#ABD9E9"
)

DimPlot(
  MCF7_Merge,
  group.by = "sample",
  cols = my_colors,
  reduction = "umap",
) +
  ggtitle("UMAP of MCF7 samples") +
  theme_classic(base_size = 14)

DimPlot(
  MCF7_Merge,
  group.by = "sample",
  cols = my_colors,
  reduction = "tsne",
) +
  ggtitle("tsne of MCF7 samples") +
  theme_classic(base_size = 14)
saveRDS(MCF7_Merge, file = "../Data/HumanCelllline/MCF7_merge_exp.rds")

# generate data for visualization
MCF7_Merge_meta <- MCF7_Merge@meta.data
MCF7_Merge_meta$cell_id <- rownames(MCF7_Merge_meta)
MCF7_Merge_pca <- Embeddings(MCF7_exp, "pca")[, 1:20]
MCF7_Merge_tSNE <- as.data.frame(MCF7_Merge@reductions$tsne@cell.embeddings)
MCF7_Merge_tSNE$cell_id <- rownames(MCF7_Merge_tSNE)
MCF7_Merge_meta_tTSNE <- merge(MCF7_Merge_meta, MCF7_Merge_tSNE, by = "cell_id")
MCF7_exp_data <- list(meta = MCF7_Merge_meta_tTSNE,
                      pca = MCF7_Merge_pca)
saveRDS(MCF7_exp_data, file = "../Data/HumanCelllline/MCF7_exp_data.rds")

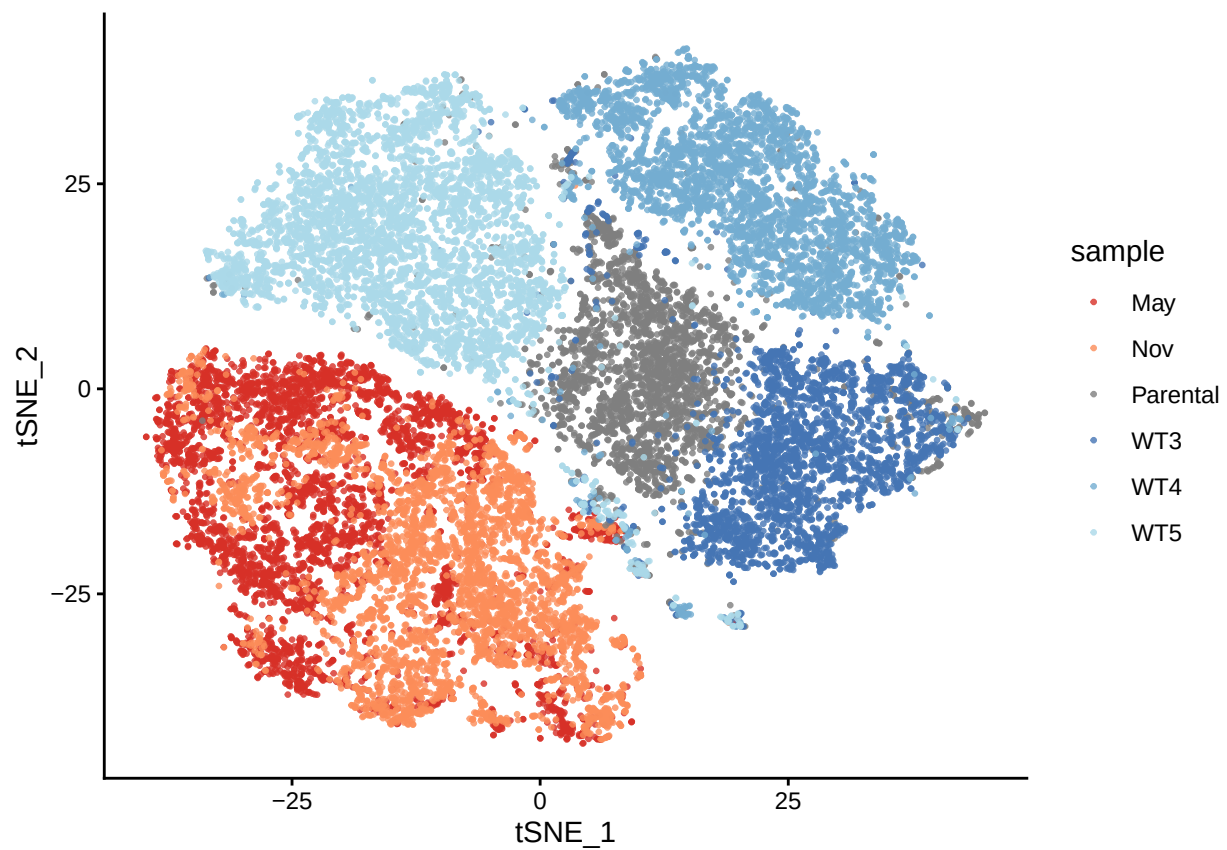
```

8 MCF7ExpSim

```
MCF7_exp <- readRDS("../Data/HumanCelllline/MCF7_exp_data.rds")

my_colors <- c(
  "May"      = "#D73027",
  "Nov"      = "#FC8D59",
  "Parental" = "#7F7F7F",
  "WT3"      = "#4575B4",
  "WT4"      = "#74ADD1",
  "WT5"      = "#ABD9E9"
)

p_tsNE <- ggplot(MCF7_exp$meta, aes(x = tSNE_1, y = tSNE_2, color = sample)) +
  geom_point(size = 0.5, alpha = 0.8) +
  scale_color_manual(values = my_colors) +
  theme_classic()
p_tsNE
```



```
# distance
pca_data <- MCF7_exp$pca
meta <- MCF7_exp$meta

avg_pca <- as.data.frame(pca_data) %>%
```

```

mutate(group = meta$orig.ident) %>%
group_by(group) %>%
summarise(across(everything(), mean))

dist_mat <- dist(avg_pca[,-1])
dist_mat <- as.matrix(dist_mat)
rownames(dist_mat) <- avg_pca$group
colnames(dist_mat) <- avg_pca$group

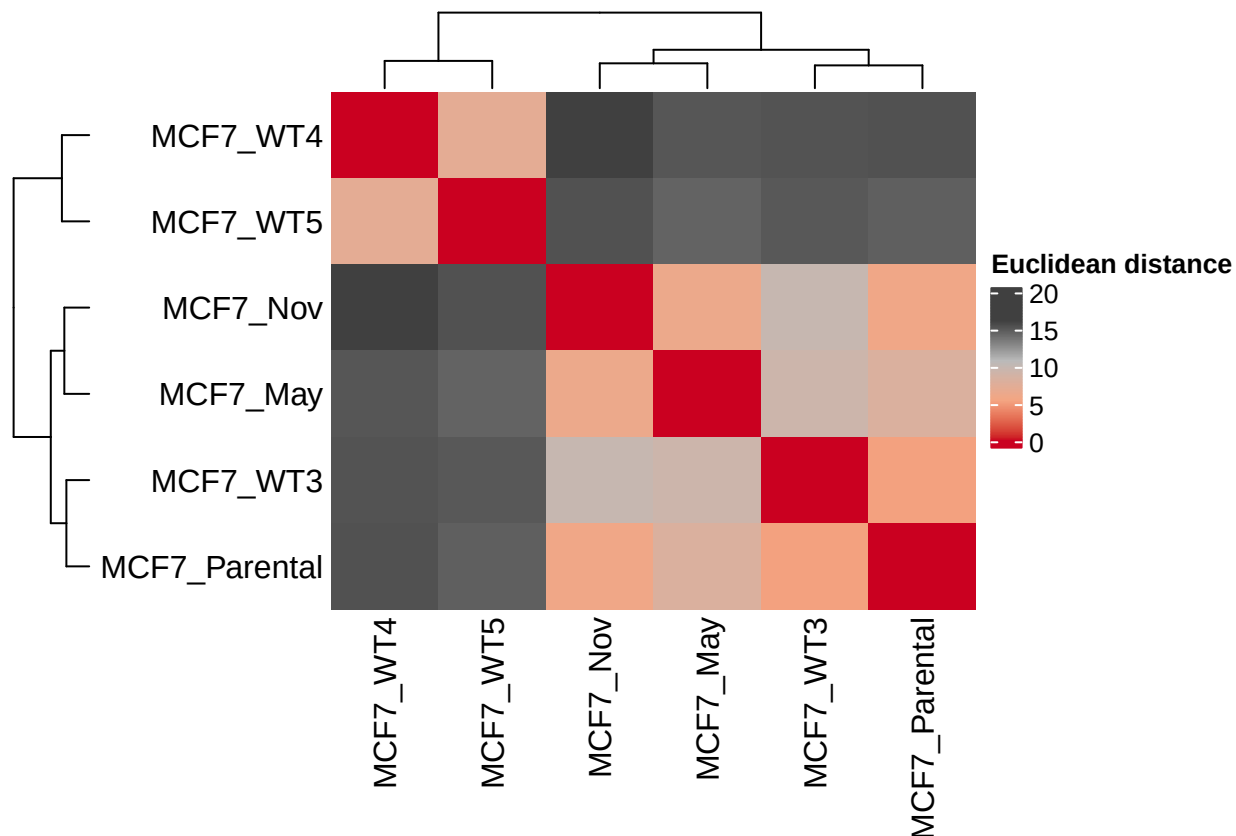
max_val <- max(dist_mat)
min_val <- min(dist_mat)
breaks <- seq(min_val, max_val, length.out = 4)

col_fun_v1 <- circlize::colorRamp2(
  breaks,
  c("#ca0020", "#f4a582", "#bababa", "#404040")
)

p_selected_MCF7_exp <- Heatmap(dist_mat,
  name = "Euclidean distance",
  col = col_fun_v1,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  show_column_names = TRUE,
  row_names_side = "left",
  column_names_side = "bottom")

p_selected_MCF7_exp

```



9 sessionInfo

```
sessionInfo()
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Stockholm
## tzcode source: internal
##
## attached base packages:
## [1] grid      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] ComplexHeatmap_2.24.1 Seurat_5.3.0          SeuratObject_5.2.0
```

```

## [4] sp_2.2-0                dplyr_1.1.4                circlize_0.4.16
## [7] ggplot2_4.0.0            pheatmap_1.0.13
##
## loaded via a namespace (and not attached):
## [1] RColorBrewer_1.1-3        rstudioapi_0.17.1          jsonlite_2.0.0
## [4] shape_1.4.6.1            magrittr_2.0.4            spatstat.utils_3.2-0
## [7] farver_2.1.2             rmarkdown_2.30           GlobalOptions_0.1.2
## [10] vctr_0.6.5              ROCR_1.0-11              Cairo_1.6-5
## [13] spatstat.explore_3.5-3    htmltools_0.5.8.1        sctransform_0.4.2
## [16] parallelly_1.45.1        KernSmooth_2.23-26       htmlwidgets_1.6.4
## [19] ica_1.0-3                plyr_1.8.9               plotly_4.11.0
## [22] zoo_1.8-14              igraph_2.1.4             mime_0.13
## [25] lifecycle_1.0.4         iterators_1.0.14         pkgconfig_2.0.3
## [28] Matrix_1.7-4            R6_2.6.1                 fastmap_1.2.0
## [31] clue_0.3-66             fitdistrplus_1.2-4       future_1.67.0
## [34] shiny_1.11.1            digest_0.6.37            colorspace_2.1-2
## [37] patchwork_1.3.2         S4Vectors_0.46.0        tensor_1.5.1
## [40] RSpectra_0.16-2         irlba_2.3.5.1           labeling_0.4.3
## [43] progressr_0.16.0        spatstat.sparse_3.1-0    httr_1.4.7
## [46] polyclip_1.10-7         abind_1.4-8              compiler_4.5.1
## [49] withr_3.0.2             doParallel_1.0.17       S7_0.2.0
## [52] fastDummies_1.7.5       MASS_7.3-65             rjson_0.2.23
## [55] tools_4.5.1            lmtest_0.9-40           httpuv_1.6.16
## [58] future.apply_1.20.0     goftest_1.2-3           glue_1.8.0
## [61] nlme_3.1-168            promises_1.3.3          Rtsne_0.17
## [64] cluster_2.1.8.1        reshape2_1.4.4          generics_0.1.4
## [67] gtable_0.3.6           spatstat.data_3.1-8     tidy_1.3.1
## [70] data.table_1.17.8       BiocGenerics_0.54.0     spatstat.geom_3.6-0
## [73] RcppAnnoy_0.0.22       ggrepel_0.9.6           RANN_2.6.2
## [76] foreach_1.5.2          pillar_1.11.1          stringr_1.5.2
## [79] spam_2.11-1            RcppHNSW_0.6.0         later_1.4.4
## [82] splines_4.5.1          lattice_0.22-7          survival_3.8-3
## [85] deldir_2.0-4           tidyselect_1.2.1        miniUI_0.1.2
## [88] pbapply_1.7-4          knitr_1.50             gridExtra_2.3
## [91] IRanges_2.42.0         scattermore_1.2         stats4_4.5.1
## [94] xfun_0.53             matrixStats_1.5.0      stringi_1.8.7
## [97] lazyeval_0.2.2         yaml_2.3.10            evaluate_1.0.5
## [100] codetools_0.2-20       tibble_3.3.0           cli_3.6.5
## [103] uwot_0.2.3            xtable_1.8-4           reticulate_1.43.0
## [106] dichromat_2.0-0.1      Rcpp_1.1.0             globals_0.18.0
## [109] spatstat.random_3.4-2  png_0.1-8              spatstat.univar_3.1-4
## [112] parallel_4.5.1         dotCall64_1.2          listen_0.9.1
## [115] viridisLite_0.4.2     scales_1.4.0           ggirdges_0.5.7
## [118] purrr_1.1.0           crayon_1.5.3           GetoptLong_1.0.5
## [121] rlang_1.1.6           cowplot_1.2.0

```